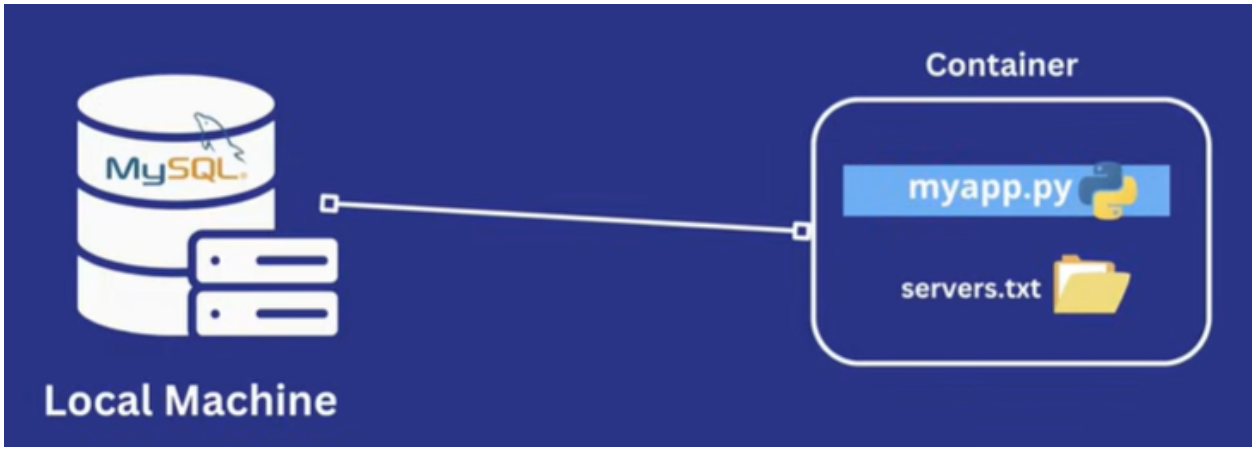
Communication Container and Local DB

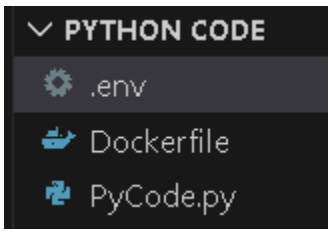


⚙️ Prerequisites

pip install mysql-connector-python

```
C:\Users\kiran\OneDrive\Desktop>pip install mysql-connect
or-python
Requirement already satisfied: mysql-connector-python in
c:\users\kiran\.virtualenvs\my_project-ymohv98n\lib\site-
packages (9.6.0)

[notice] A new release of pip is available: 25.2 -> 26.0.
1
[notice] To update, run: python.exe -m pip install --upgr
ade pip
```

Folder Structure:



Python Code:

```
import mysql.connector
from dotenv import load_dotenv
import os
```

```python
# ------------------ LOAD ENV ------------------ #
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_NAME = os.getenv("DB_NAME")

# ------------------ DB CONNECTION ------------------ #
def connect_db():
    return mysql.connector.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        auth_plugin='mysql_native_password'
    )

def setup_database():
    conn = connect_db()
    cursor = conn.cursor()

    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    cursor.execute(f"USE {DB_NAME}")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100),
            phone VARCHAR(15) NOT NULL,
            salary FLOAT,
            aptitude_score FLOAT NOT NULL
        )
    """)

    conn.commit()
    return conn, cursor

# ------------------ ADD EMPLOYEE ------------------ #
def add_employee(cursor, conn):
    print("\n--- Add Employee ---")

    name = input("Enter Name (required): ").strip()
    phone = input("Enter Phone (required): ").strip()
```

```
    email = input("Enter Email (optional): ").strip()
    salary = input("Enter Salary (optional): ").strip()
    aptitude = input("Enter Aptitude Score (0-10) (required): ").strip()

    if not name or not phone or not aptitude:
        print("❌ Employee NOT added. Required fields missing.\n")
        return

    try:
        salary = float(salary) if salary else None
        aptitude = float(aptitude)

        if aptitude < 0 or aptitude > 10:
            print("❌ Aptitude score must be between 0 and 10.\n")
            return

        cursor.execute("""
            INSERT INTO employees (name, email, phone, salary, aptitude_score)
            VALUES (%s, %s, %s, %s, %s)
        """, (name, email if email else None, phone, salary, aptitude))

        conn.commit()
        print("✅ Employee added successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ UPDATE EMPLOYEE ------------------ #
def update_employee(cursor, conn):
    print("\n--- Update Employee ---")
    emp_id = input("Enter Employee ID: ").strip()

    if not emp_id:
        print("❌ ID not provided. Try again.\n")
        return

    try:
        cursor.execute("SELECT * FROM employees WHERE id=%s", (emp_id,))
        if not cursor.fetchone():
            print("❌ Employee not found.\n")
            return

        print("""
        a. Update Name
```

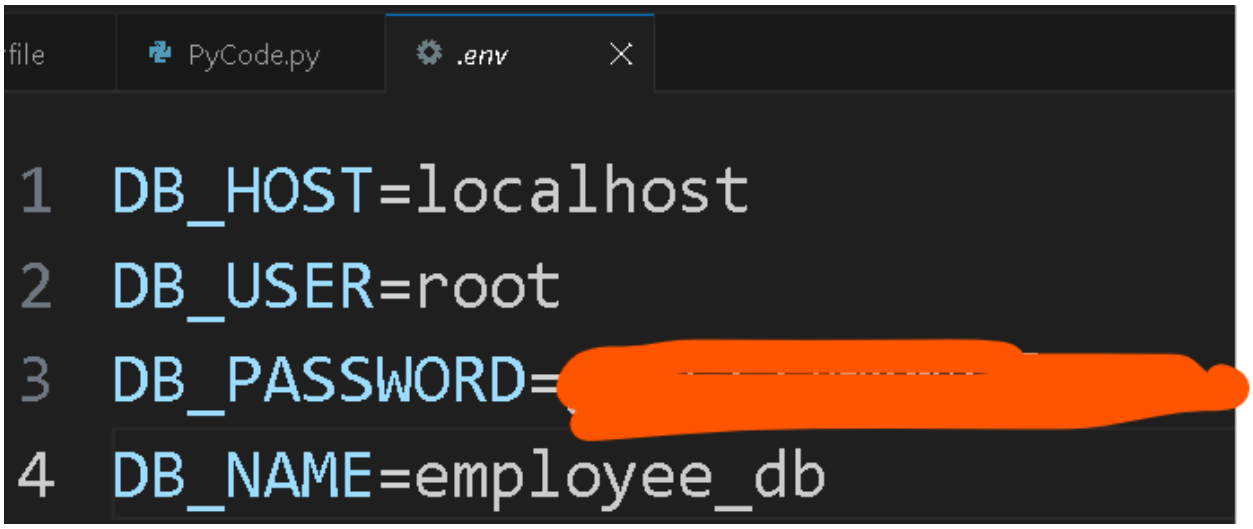
```
    b. Update Email
    c. Update Phone
    d. Update Salary
    e. Update Aptitude Score
    """)

    choice = input("Choose option: ").lower()

    field_map = {
        'a': 'name',
        'b': 'email',
        'c': 'phone',
        'd': 'salary',
        'e': 'aptitude_score'
    }

    if choice not in field_map:
        print("❌ Invalid choice.\n")
        return

    new_value = input("Enter new value: ").strip()

    if not new_value:
        print("❌ Value cannot be empty.\n")
        return

    # Type handling
    if field_map[choice] == 'salary':
        new_value = float(new_value)
    elif field_map[choice] == 'aptitude_score':
        new_value = float(new_value)
        if new_value < 0 or new_value > 10:
            print("❌ Aptitude must be between 0 and 10.\n")
            return

    query = f"UPDATE employees SET {field_map[choice]}=%s WHERE id=%s"
    cursor.execute(query, (new_value, emp_id))
    conn.commit()

    print("✅ Employee updated successfully!\n")

except Exception as e:
    print("❌ Error:", e)
```

```
# ------------------ VIEW EMPLOYEES ------------------ #
def view_employees(cursor):
    print("\n--- View Employees ---")
    choice = input("Show all? (y/n): ").lower()

    try:
        if choice == 'y':
            cursor.execute("SELECT * FROM employees")
        else:
            ids = input("Enter min_id max_id: ").split()
            if len(ids) != 2:
                print("❌ Invalid input.\n")
                return

            min_id, max_id = ids
            cursor.execute(
                "SELECT * FROM employees WHERE id BETWEEN %s AND %s",
                (min_id, max_id)
            )

        rows = cursor.fetchall()

        if not rows:
            print("⚠️ No employees found.\n")
            return

        print("\n--- Employee Records ---")
        for row in rows:
            print(row)
        print()

    except Exception as e:
        print("❌ Error:", e)

# ------------------ DELETE EMPLOYEE ------------------ #
def delete_employee(cursor, conn):
    print("\n--- Delete Employee ---")
    emp_id = input("Enter Employee ID to delete: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
```

```
        cursor.execute("DELETE FROM employees WHERE id=%s", (emp_id,))
        conn.commit()

        if cursor.rowcount == 0:
            print("❌ Employee not found.\n")
        else:
            print("✅ Employee deleted successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ MAIN LOOP ------------------ #
def main():
    try:
        conn, cursor = setup_database()
        print("✅ Database connected successfully!\n")

        while True:
            print("""
===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
""")

            choice = input("Enter choice (1-5): ").strip()

            if choice == '1':
                add_employee(cursor, conn)
            elif choice == '2':
                update_employee(cursor, conn)
            elif choice == '3':
                view_employees(cursor)
            elif choice == '4':
                delete_employee(cursor, conn)
            elif choice == '5':
                print("👋 Exiting program...")
                break
            else:
                print("❌ Invalid input. Please enter 1-5.\n")

        cursor.close()
```

```
        conn.close()

    except Exception as e:
        print("❌ Database connection failed:", e)

# ------------------ RUN ------------------ #
if __name__ == "__main__":
    main()
```

.env file:

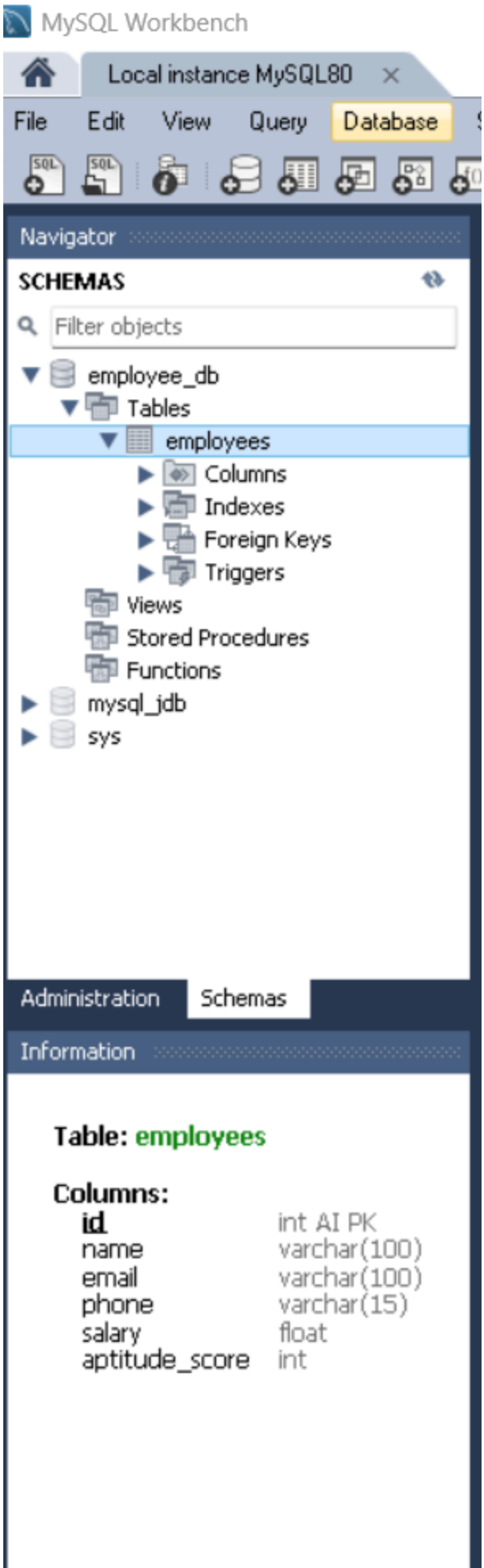


Execute:

```
C:\Users\kiran\OneDrive\Desktop\Python Code>python PyCode.py
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): routh kiran babu
Enter Phone (required): 234231
Enter Email (optional): sfa@2323.com
Enter Salary (optional): 241123
Enter Aptitude Score (0-10) (required): 7.452
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
```

```
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 3

--- View Employees ---
Show all? (y/n): y

--- Employee Records ---
(2, 'routh kiran babu', 'sfa@2323.com', '234231', 241123.0, 7)


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5):
```

In MySQL Workbench: it automatically adds it. Without out manually creating database and table.


MySQL Workbench
Local instance MySQL80
File
Edit
View
Query
Database
Navigator
SCHEMAS
Filter objects
employee_db
Tables
employees
Columns
Indexes
Foreign Keys
Triggers
Views
Stored Procedures
Functions
mysql_jdb
sys
Administration
Schemas
Information
Table: employees
Columns:
id int AI PK
name varchar(100)
email varchar(100)
phone varchar(15)
salary float
aptitude_score int

```
--- Employee Records ---
(2, 'routh kiran babu', 'sfa@2323.com', '234231', 241123.0, 7)
(3, 'new name', '2342.2342@32.come', '23234', 234234.0, 4)
(4, 'mame3', 'wer.dfdf@adf.com', '3452342', 234234.0, 5)
(5, 'mame4', 'fadf@adf.com', '232424', 23434.0, 8)
(6, 'name5', 'fadf@dfa.gmail.co', '23234', 23423.0, 6)


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 5
👋 Exiting program...

C:\Users\kiran\OneDrive\Desktop\Python Code>
```

In MySQL Workbench:

Query 1

Limit to 1000 rows

```
use employee_db;
select * from employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	name	email	phone	salary	aptitude_score
2	routh kiran babu	sfa@2323.com	234231	241123	7
3	new name	2342.2342@32.come	23234	234234	4
4	mame3	wer.dfdf@adf.com	3452342	234234	5
5	mame4	fadf@adf.com	232424	23434	8
6	name5	fadf@dfa.gmail.co	23234	23423	6
NULL	NULL	NULL	NULL	NULL	NULL

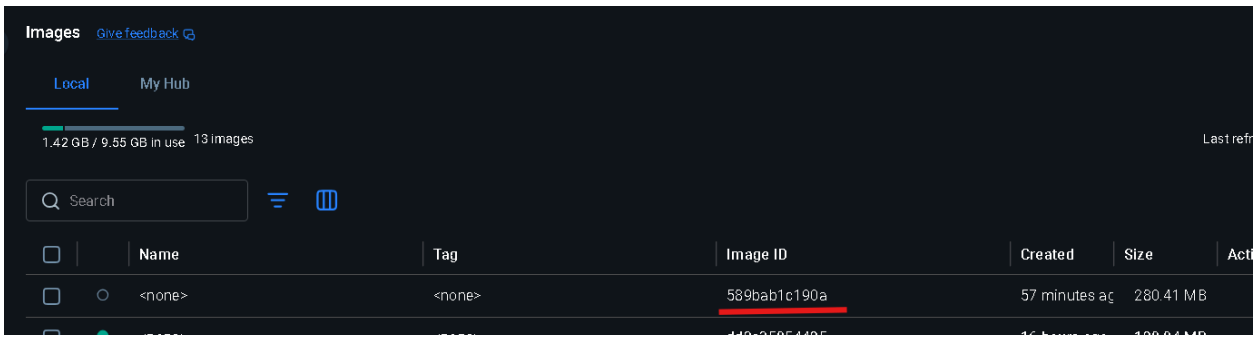
Output at terminal:

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker build .
[+] Building 33.7s (11/11) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 208B
 => [internal] load metadata for docker.io/library/python:3.10
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
```

In terminal:

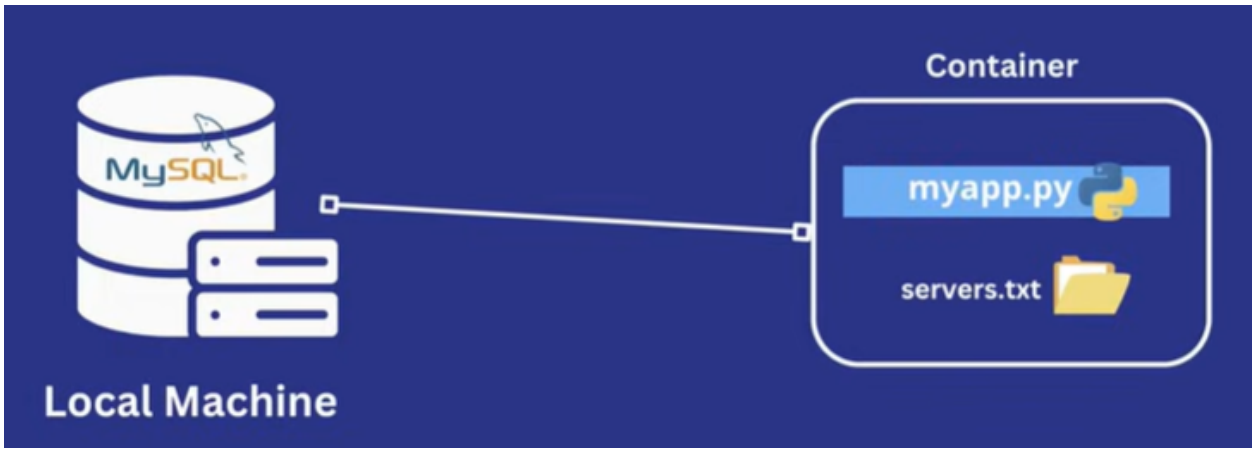
```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker images -a

IMAGE                                ID            DISK USAGE   CONTENT SIZE   EXTRA
java-input-app:latest                2ee3d21b2faa       636MB         198MB
mywebapp:01                          89eacf391531      2.11GB         491MB
mywebapp:02                          b90c1a14ca28      2.11GB         491MB
nginx:latest                         dec7a90bd097       240MB        65.8MB
openjdk:27-ea-trixie                 4a7da6fcd5de      1.25GB         396MB
python:latest                        0040600478fe      1.63GB         432MB
routhkiranbabu/webapp-demo:01        7023f47e1922      2.11GB         491MB
routhkiranbabu/webapp-demo:02        b90c1a14ca28      2.11GB         491MB
<untagged>                           d209b8ea96db      1.61GB         414MB
<untagged>                           b9cbed6a730c      1.61GB         414MB    U
<untagged>                           589bab1c190a       280MB        67.7MB
```

In docker desktop:



After running the image:

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker run -it --rm 589bab1c190a
❌ Database connection failed: 2003 (HY000): Can't connect to MySQL server on 'localhost:3306' (111)
PS C:\Users\kiran\OneDrive\Desktop\Python Code>
```



Python code running in container, since container is isolated environment and not having any relation with os, then how it will understand localhost,
So change in .env file:

requirements.txt PyCode.py .env

```
DB_HOST=host.docker.internal
DB_USER=root
DB_PASSWORD=[redacted]
DB_NAME=employee_db
```

So finally:

Folder structure:

PYTHON CODE
- .env
- Dockerfile
- PyCode.py
- requirements.txt

Dockerfile has:

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY PyCode.py .

CMD ["python", "PyCode.py"]
```

Requirements.txt has:

mysql-connector-python==8.4.0
python-dotenv==1.0.1

.env file has:

```
DB_HOST=host.docker.internal
DB_USER=root
DB_PASSWORD=[redacted]
DB_NAME=employee_db
```

PyCode.py has:

```
import mysql.connector
from dotenv import load_dotenv
import os

# ------------------ LOAD ENV ------------------ #
load_dotenv()

DB_HOST = os.getenv("DB_HOST")
DB_USER = os.getenv("DB_USER")
DB_PASSWORD = os.getenv("DB_PASSWORD")
DB_NAME = os.getenv("DB_NAME")

# ------------------ DB CONNECTION ------------------ #
def connect_db():
    return mysql.connector.connect(
        host=DB_HOST,
        user=DB_USER,
        password=DB_PASSWORD,
        auth_plugin='mysql_native_password'
    )

def setup_database():
    conn = connect_db()
    cursor = conn.cursor()

    cursor.execute(f"CREATE DATABASE IF NOT EXISTS {DB_NAME}")
    cursor.execute(f"USE {DB_NAME}")

    cursor.execute("""
        CREATE TABLE IF NOT EXISTS employees (
            id INT AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(100) NOT NULL,
            email VARCHAR(100),
```

```
        phone VARCHAR(15) NOT NULL,
        salary FLOAT,
        aptitude_score FLOAT NOT NULL
    )
""")

    conn.commit()
    return conn, cursor

# ------------------ ADD EMPLOYEE ------------------ #
def add_employee(cursor, conn):
    print("\n--- Add Employee ---")

    name = input("Enter Name (required): ").strip()
    phone = input("Enter Phone (required): ").strip()
    email = input("Enter Email (optional): ").strip()
    salary = input("Enter Salary (optional): ").strip()
    aptitude = input("Enter Aptitude Score (0-10) (required): ").strip()

    if not name or not phone or not aptitude:
        print("❌ Employee NOT added. Required fields missing.\n")
        return

    try:
        salary = float(salary) if salary else None
        aptitude = float(aptitude)

        if aptitude < 0 or aptitude > 10:
            print("❌ Aptitude score must be between 0 and 10.\n")
            return

        cursor.execute("""
            INSERT INTO employees (name, email, phone, salary, aptitude_score)
            VALUES (%s, %s, %s, %s, %s)
        """, (name, email if email else None, phone, salary, aptitude))

        conn.commit()
        print("✅ Employee added successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ UPDATE EMPLOYEE ------------------ #
def update_employee(cursor, conn):
```

```
print("\n--- Update Employee ---")
emp_id = input("Enter Employee ID: ").strip()

if not emp_id:
    print("❌ ID not provided. Try again.\n")
    return

try:
    cursor.execute("SELECT * FROM employees WHERE id=%s", (emp_id,))
    if not cursor.fetchone():
        print("❌ Employee not found.\n")
        return

    print("""
    a. Update Name
    b. Update Email
    c. Update Phone
    d. Update Salary
    e. Update Aptitude Score
    """)

    choice = input("Choose option: ").lower()

    field_map = {
        'a': 'name',
        'b': 'email',
        'c': 'phone',
        'd': 'salary',
        'e': 'aptitude_score'
    }

    if choice not in field_map:
        print("❌ Invalid choice.\n")
        return

    new_value = input("Enter new value: ").strip()

    if not new_value:
        print("❌ Value cannot be empty.\n")
        return

    # Type handling
    if field_map[choice] == 'salary':
        new_value = float(new_value)
```

```
        elif field_map[choice] == 'aptitude_score':
            new_value = float(new_value)
            if new_value < 0 or new_value > 10:
                print("❌ Aptitude must be between 0 and 10.\n")
                return

        query = f"UPDATE employees SET {field_map[choice]}=%s WHERE id=%s"
        cursor.execute(query, (new_value, emp_id))
        conn.commit()

        print("✅ Employee updated successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ VIEW EMPLOYEES ------------------ #
def view_employees(cursor):
    print("\n--- View Employees ---")
    choice = input("Show all? (y/n): ").lower()

    try:
        if choice == 'y':
            cursor.execute("SELECT * FROM employees")
        else:
            ids = input("Enter min_id max_id: ").split()
            if len(ids) != 2:
                print("❌ Invalid input.\n")
                return

            min_id, max_id = ids
            cursor.execute(
                "SELECT * FROM employees WHERE id BETWEEN %s AND %s",
                (min_id, max_id)
            )

        rows = cursor.fetchall()

        if not rows:
            print("⚠️ No employees found.\n")
            return

        print("\n--- Employee Records ---")
        for row in rows:
            print(row)
```

```
        print()

    except Exception as e:
        print("❌ Error:", e)

# ------------------ DELETE EMPLOYEE ------------------ #
def delete_employee(cursor, conn):
    print("\n--- Delete Employee ---")
    emp_id = input("Enter Employee ID to delete: ").strip()

    if not emp_id:
        print("❌ ID not provided.\n")
        return

    try:
        cursor.execute("DELETE FROM employees WHERE id=%s", (emp_id,))
        conn.commit()

        if cursor.rowcount == 0:
            print("❌ Employee not found.\n")
        else:
            print("✅ Employee deleted successfully!\n")

    except Exception as e:
        print("❌ Error:", e)

# ------------------ MAIN LOOP ------------------ #
def main():
    try:
        conn, cursor = setup_database()
        print("✅ Database connected successfully!\n")

        while True:
            print("""
===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit
""")

            choice = input("Enter choice (1-5): ").strip()
```

```
            if choice == '1':
                add_employee(cursor, conn)
            elif choice == '2':
                update_employee(cursor, conn)
            elif choice == '3':
                view_employees(cursor)
            elif choice == '4':
                delete_employee(cursor, conn)
            elif choice == '5':
                print("👋 Exiting program...")
                break
            else:
                print("❌ Invalid input. Please enter 1-5.\n")

        cursor.close()
        conn.close()

    except Exception as e:
        print("❌ Database connection failed:", e)

# ------------------ RUN ------------------ #
if __name__ == "__main__":
    main()
```

To run the code:

To build:

```
p\Python Code> docker build .
D
ion from Dockerfile
08B
docker.io/library/python:3.10-slim
ken for registry-1.docker.io
```

```
sktop\Python Code> docker images -a

   ID              DISK USAGE   CONTE
   2ee3d21b2faa        636MB
   89eacf391531       2.11GB
   b90c1a14ca28       2.11GB
   dec7a90bd097        240MB
   4a7da6fcd5de       1.25GB
   0040600478fe       1.63GB
   7023f47e1922       2.11GB
   b90c1a14ca28       2.11GB
   b9cbed6a730c       1.61GB
   28c1f5355aff        280MB
   26366c12d2e4       1.61GB
   dd2e35854495        199MB
   84cac96bdf12        280MB
```

```
\Python Code> docker run -it --rm --env-file .env 84cac96bdf12
```

```
PS C:\Users\kiran\OneDrive\Desktop\Python Code> docker run -it --rm --env-file .env 84cac96bdf12
✅ Database connected successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): routh kiran babu
Enter Phone (required): 23423414
Enter Email (optional): 234@gam.com
Enter Salary (optional): 341234
Enter Aptitude Score (0-10) (required): 4.242
✅ Employee added successfully!
```

```
Enter choice (1-5): 1

--- Add Employee ---
Enter Name (required): routh name1
Enter Phone (required): 234234
Enter Email (optional): 3423@gaa.com
Enter Salary (optional): 2323
Enter Aptitude Score (0-10) (required): 7.342
✅ Employee added successfully!


===== Employee Management =====
1. Add Employee
2. Update Employee
3. Show Employees
4. Delete Employee
5. Quit

Enter choice (1-5): 3

--- View Employees ---
Show all? (y/n): y

--- Employee Records ---
(2, 'routh kiran babu', 'sfa@2323.com', '234231', 241123.0, 7)
(3, 'new name', '2342.2342@32.come', '23234', 234234.0, 4)
(4, 'mame3', 'wer.dfdf@adf.com', '3452342', 234234.0, 5)
(5, 'mame4', 'fadf@adf.com', '232424', 23434.0, 8)
(4, 'mame3', 'wer.dfdf@adf.com', '3452342', 234234.0, 5)
(4, 'mame3', 'wer.dfdf@adf.com', '3452342', 234234.0, 5)
(4, 'mame3', 'wer.dfdf@adf.com', '3452342', 234234.0, 5)
(5, 'mame4', 'fadf@adf.com', '232424', 23434.0, 8)
(6, 'name5', 'fadf@dfa.gmail.co', '23234', 23423.0, 6)
(7, 'routh kiran babu', '234@gam.com', '23423414', 341234.0, 4)
(8, 'routh name1', '3423@gaa.com', '234234', 2323.0, 7)
```

Query 1

Limit to 1000 rows

```
use employee_db;
select * from employees;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

id	name	email	phone	salary	aptitude_score
2	routh kiran babu	sfa@2323.com	234231	241123	7
3	new name	2342.2342@32.come	23234	234234	4
4	mame3	wer.dfdf@adf.com	3452342	234234	5
5	mame4	fadf@adf.com	232424	23434	8
6	name5	fadf@dfa.gmail.co	23234	23423	6
7	routh kiran babu	234@gam.com	23423414	341234	4
8	routh name1	3423@gaa.com	234234	2323	7
NULL	NULL	NULL	NULL	NULL	NULL

From inside the container, we can modify database in real time.